



Project Report on *“Image to Sketch”* Using U-Net CNN

Prepared By:

Satwik Shreshth
MCA 3rd Sem

Roll: 24MCA014

Submitted to:

Prof. (Dr.) Ratika Pradhan
Department of Computer Applications
Sikkim University

Abstract

This project develops and implements a **U-Net convolutional neural network (CNN)** model to convert RGB photographs into grayscale pencil-like sketches. The dataset used consists of paired photo and sketch images stored in Parquet format. We preprocess this data by decoding, resizing, and normalizing the images into numpy arrays suitable for model training. The dataset is split into training and testing sets with an 80:20 ratio.

The U-Net architecture consists of encoder and decoder blocks with skip connections, enabling the model to capture and reconstruct detailed structural features from input photos. A combined loss function including Mean Absolute Error (MAE) and Structural Similarity Index Measure (SSIM) is employed to enhance pixel-level accuracy and structural similarity of the generated sketches. The model is compiled with the Adam optimizer and trained for 50 epochs with batch size 16.

Evaluation is conducted on the test set using multiple metrics *i.e.* MAE, Mean Squared Error (MSE), SSIM, and Peak Signal-to-Noise Ratio (PSNR) demonstrating the model's effectiveness in producing visually coherent and accurate sketches. Visual comparisons between input photos, ground truth sketches, and model predictions further validate the approach.

The trained model is saved for reuse and deployed in a simple Streamlit web application, allowing users to upload photos and obtain sketch predictions interactively. This project highlights the potential of deep encoder-decoder networks like U-Net for artistic-style image translation in practical digital art and computer vision applications.

Table of Contents

Chapter	Page No.
1. Introdcction	4
2. Methodology	5
3. Code Explanation	11
4. Result and Discussion	14
5. Conclusion and Future Work	15
6. Refrences	16\

Introduction

Computer vision is a field of artificial intelligence that enables machines to interpret and understand visual data similar to human vision. It involves methods for processing, analyzing, and extracting meaningful information from images or videos. One important application of computer vision is image-to-image translation, where an input image is transformed into another form, such as converting a photo into a sketch.

This project focuses on developing a deep learning model based on the UNet convolutional neural network (CNN) architecture for photo-to-sketch conversion. U-Net is known for its encoder-decoder structure with skip connections, which allows the model to capture both global and fine details in the image. Training the model with a combined loss function that includes Mean Absolute Error (MAE) and Structural Similarity Index Measure (SSIM) enhances the accuracy and visual quality of the generated sketches.

The dataset consists of paired photographs and sketches, which are preprocessed and split into training and testing sets. The model is then trained and evaluated on this dataset. This work demonstrates how U-Net CNNs can be effectively applied to artistic and practical image transformation tasks, providing a foundation for further research and applications in digital art and computer vision.

Methodology

Model Architecture and Justification:

The model employed for this project is based on the U-Net convolutional neural network architecture, which has proven highly effective for image-to-image translation tasks. The U-Net follows a distinct U-shaped design comprising two main parts: an encoder (contracting path) and a decoder (expanding path).

The encoder progressively extracts image features through successive convolutional layers followed by max-pooling, which reduces the spatial dimensions while increasing feature depth. This process captures hierarchical, contextual information from the input images. At the deepest layer lies the bottleneck, which holds the most compressed and abstract representation of the input features.

Complementing the encoder, the decoder gradually recovers spatial resolution by upsampling the feature maps using transpose convolutions. It reconstructs the output image, layer by layer, with the goal of generating a sketch that closely resembles the input photo's structure.

A critical component distinguishing U-Net from traditional CNNs is the extensive use of skip connections. These connections link each encoder layer to its corresponding decoder layer and concatenate their feature maps. This mechanism enables the model to combine the abstract, semantic information from the encoder with fine-grained spatial details preserved from earlier layers, mitigating the loss of resolution typically caused by downsampling in regular CNNs. This is essential for precise localization and generating detailed sketches.

Why U-Net Was Chosen Over Standard CNNs:

In earlier experiments, a standard CNN was utilized for sketch reconstruction. While the model captured general features, it underperformed in

preserving the structural and textural intricacies of sketches. The absence of skip connections meant that spatial details diminished as images passed through pooling layers, leading to blurred and less accurate outputs.

U-Net's architecture directly addresses these shortcomings. The skip connections ensure that critical low-level details are never lost and are effectively combined with high-level features during reconstruction. This results in clearer, more structurally coherent sketches compared to traditional CNN outputs.

Furthermore, the symmetrical encoder-decoder framework of U-Net balances the extraction of global context with local detail preservation, which is uniquely suited to tasks where pixel-wise accuracy and perceptual quality are both vital. U-Net's demonstrated success in biomedical image segmentation and various image translation problems reinforces its appropriateness.

In summary, U-Net's design offers superior capacity to maintain spatial hierarchy and fine details through skip connections and encoder-decoder symmetry, making it an ideal choice for photo-to-sketch conversion over more simplistic CNN architectures.

How U-Net Works:

The U-Net architecture processes images through two primary paths: an encoder and a decoder that work symmetrically to achieve precise image segmentation or translation. First, the input image, usually grayscale or RGB, is fed into the encoder. The encoder's function is to extract important features by applying successive convolutional layers that detect patterns and textures. After each set of convolutions, max-pooling is used to reduce the spatial dimensions while increasing the number of feature channels. This process allows the network to learn increasingly complex, abstract representations of the image as it goes deeper.

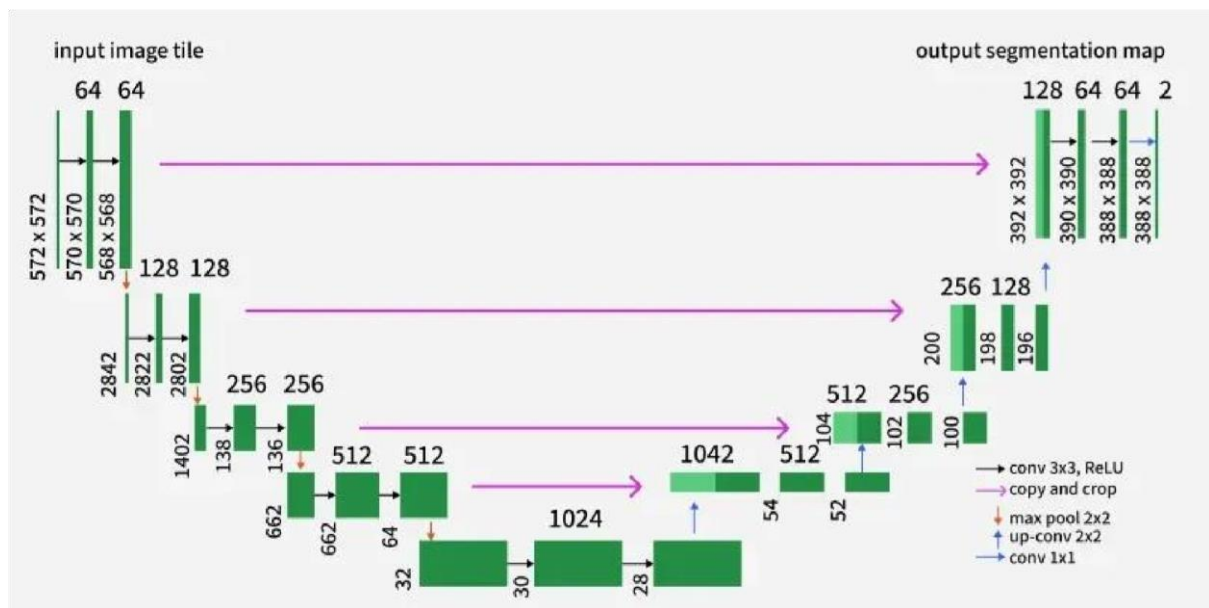
The network then reaches a bottleneck layer, which contains the most compressed form of the input image's features. This layer holds the essential information needed for reconstruction.

Following the bottleneck is the decoder. The decoder gradually upsamples the compressed features to restore the image to its original size. At each step of this upsampling, the decoder combines the current feature map with the feature map from the corresponding encoder layer through skip connections. These skip connections are crucial because they pass along fine spatial details that might

have been lost during downsampling, helping the decoder maintain accuracy when reconstructing edges and small structures.

Finally, a 1 by 1 convolutional layer is applied to the output of the decoder to produce the final segmentation map or in this case, the grayscale sketch. This layer reduces the number of feature channels to match the desired output, classifying each pixel appropriately and ensuring the output image has the same spatial resolution as the input.

This detailed flow allows U-Net to capture both global context and local details, making it highly effective for image-to-image tasks compared to traditional CNNs.



Dataset Description:

The dataset used in this study consists of paired images of photographs and their corresponding sketches. The data was stored in Parquet file format containing columns for photo and sketch images. Each pair forms an input-output sample for supervised learning. The dataset provides sufficient variety in scenes and styles to train a robust model.

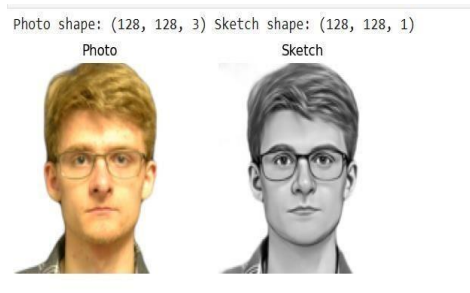


Image 1: This figure shows a pair of images from the dataset: the left image is a color photo resized to 128 by 128 pixels, and the right image is its corresponding grayscale sketch, both used to train and evaluate the photo-to-sketch conversion model.

Data Preprocessing

The raw images were extracted from the Parquet file and decoded into array formats. Both photos and sketches were resized to 128×128 pixels to standardize input dimensions across samples. Photos were normalized to have pixel values between 0 and 1 to facilitate training stability. Sketches were converted to grayscale before normalization to match the expected single-channel output format. The processed images were stored as NumPy arrays. Further, the dataset was split into training (80%) and testing (20%) subsets ensuring the model's performance could be evaluated on unseen data.

```
Dataset shapes: (76, 128, 128, 3) (76, 128, 128, 1)
```

Image 2: This result shows that all photo–sketch pairs from the dataset were successfully converted into numpy arrays of matching dimensions, ready for use in CNN model training.

```
Training set: (60, 128, 128, 3) (60, 128, 128, 1)
Testing set: (16, 128, 128, 3) (16, 128, 128, 1)
```

Image 3: This split shows that the dataset of photo-sketch pairs was divided into 60 samples for training and 16 samples for testing, with each set formatted as numpy arrays ready for model development.

Architecture:

A U-Net architecture was implemented to perform image-to-sketch translation. The network comprised an encoder path with consecutive convolutional layers followed by max-pooling for feature extraction and downsampling. The bottleneck layer captured high-level abstract features. This was succeeded by a decoder path featuring transposed convolutional layers that

upsample and reconstruct the output sketch. The encoder and decoder blocks are connected via skip connections, allowing preservation of spatial details critical for high-quality sketch generation. The final output layer used a sigmoid activation function to produce normalized grayscale images.

Loss Functions and Evaluation Metrics:

The model was trained using a combined loss function incorporating Mean Absolute Error (MAE) and Structural Similarity Index Measure (SSIM) loss. MAE minimized pixel-wise differences between predicted and true sketches, while SSIM loss optimized the preservation of structural details and perceptual quality. Favoring both objectives enhanced sketch realism beyond traditional pixel-based losses. Evaluation metrics applied on test data included MAE, Mean Squared Error (MSE), SSIM, and Peak Signal-to-Noise Ratio (PSNR), providing a comprehensive assessment of reconstruction quality and structural fidelity.

```
Test Set Performance:
Average MAE: 0.06621605
Average MSE: 0.01945625
Average SSIM: 0.7196740360298697
Average PSNR: 18.100567680449267
```

Image 4: This result summarizes the test set performance of the model using key evaluation metrics: MAE, MSE, SSIM, and PSNR.

Training Procedure:

The U-Net model was compiled with the Adam optimizer and trained for 50 epochs using batches of 16 samples per iteration. The validation set allowed monitoring generalization performance during training. Model checkpoints and summaries guided hyperparameter tuning and debugging.

Testing and Deployment:

Post-training, the model was tested on the holdout test set. Quantitative results from evaluation metrics were computed for all test samples. Visualizations comparing original photos, ground truth sketches, and model predictions illustrated qualitative performance. Additionally, the trained model was saved and integrated into a Streamlit web application allowing users to upload photos and receive instant sketch output, demonstrating practical utility.



Image 5: This comparison shows an input photo, its true sketch, and the model's predicted sketch, demonstrating how closely the generated sketch matches the ground truth.



Image 6: This screenshot shows the Streamlit interface of the Photo to Sketch Generator, where a user uploads a photo and receives the corresponding predicted sketch created by the U-Net CNN model.

Code Explanation

1. Hyperparameters Choice

IMG_SIZE = 128

BATCH_SIZE = 16

EPOCHS = 50

LEARNING_RATE = 0.001

I chose an image size (IMG_SIZE) of 128 because it offers a good balance between preserving enough visual detail for sketch generation and keeping computation manageable. For BATCH_SIZE, I use 16 as it fits comfortably in GPU memory during training, allowing for stable optimization without overwhelming available hardware. I set EPOCHS to 50 because deep learning models like U-Net typically require many iterations to converge, and this number provides enough time to monitor performance and stop early if overfitting is detected. My learning rate (LEARNING_RATE) is set to 0.001 since it's a standard starting point for Adam optimizer; it provides steady learning while minimizing the risk of overshooting minima or plateauing too quickly.

2. Encoder Blocks

```
def encoder_block(input_tensor,num_filters):  
x=Conv2D(num_filters,3,padding='same',activation='relu')(input_tensor) x =  
Conv2D(num_filters, 3, padding='same', activation='relu')(x)  
p = MaxPooling2D((2,2))(x) return x, p
```

In designing the encoder blocks, I use two convolutional layers per block because stacking helps the network learn more complex features at each level. The num_filters parameter increases at deeper layers, typically doubling with each block (e.g., 64, 128, 256), as deeper representations should capture more abstract patterns. The kernel size 3x3 is a well-established size for local pattern detection. Max pooling with a pool size of 2x2 is used to systematically halve the spatial dimension, focusing the model on the most dominant features while reducing computational cost.

3. Decoder Blocks def decoder_block(input_tensor, concat_tensor, num_filters):

```

    x = Conv2DTranspose(num_filters, 2, strides=(2,2)
padding='same')(input_tensor)    x = Concatenate()([x, concat_tensor])
    x = Conv2D(num_filters, 3, padding='same', activation='relu')(x)
    x = Conv2D(num_filters, 3, padding='same', activation='relu')(x)
    return x

```

I use transpose convolutions for upsampling because they are effective at reconstructing higher-resolution images and are standard in image generation architectures. The strides=(2,2) doubles the spatial size with each upsampling step. Concatenation with encoder layers (skip connections) allows the decoder to access fine spatial details lost during pooling, crucial for tasks like sketch generation that require pixel-level accuracy. Using two convolutional layers after concatenation ensures that the merged features are well-integrated and refined.

U-Net Assembly:

```

inputs = Input((IMG_SIZE, IMG_SIZE, 3)) enc1, pool1 =
encoder_block(inputs, 64) enc2, pool2 = encoder_block(pool1, 128)
enc3, pool3 = encoder_block(pool2, 256) bottleneck = Conv2D(512,
3, padding='same', activation='relu')(pool3)

```

```

dec3 = decoder_block(bottleneck, enc3, 256) dec2
= decoder_block(dec3, enc2, 128) dec1 =
decoder_block(dec2, enc1, 64) outputs =
Conv2D(1, 1, activation='sigmoid')(dec1) model =
Model(inputs, outputs)

```

I start with three encoder blocks and three decoder blocks as this depth has proven effective in various research and balances network complexity. The bottleneck uses 512 filters; this high dimensionality allows deep semantic feature representation. I chose a final 1x1 convolution with sigmoid activation to reduce

the output to a single grayscale channel, which matches the format of my sketch images.

5. **Training** **Process**

```
model.compile(optimizer=Adam(learning_rate=LEARNING_RATE),  
              loss=['mae', ssim_loss], metrics=['accuracy','mse','mae']) history  
= model.fit(X_train, Y_train, batch_size=BATCH_SIZE,  
epochs=EPOCHS,  
            validation_data=(X_test, Y_test))
```

I use the Adam optimizer for its adaptive learning rate and robustness with sparse gradients. For the loss function, I combine Mean Absolute Error (mae) and a custom Structural Similarity Index (ssim_loss). MAE encourages pixel-level closeness, while SSIM emphasizes perceptual and structural similarity between generated and true sketches. Adding accuracy, MSE, and MAE metrics helps me monitor both prediction errors and image similarity during training.

Results and Discussion

The U-Net model was evaluated on the test set after 50 epochs of training. The quantitative performance metrics for the test set are as follows:

Mean Absolute Error (MAE): 0.0662

Mean Squared Error (MSE): 0.0195

Structural Similarity Index Measure (SSIM): 0.7197

Peak Signal-to-Noise Ratio (PSNR): 18.10

The low MAE and MSE values indicate that the model predicts sketches closely matching the ground truth at the pixel level with minimal error. The SSIM score of approximately 0.72 shows good preservation of structural and textural information in the generated sketches, reflecting perceptual similarity with the original sketches. The PSNR value further supports the quality by measuring the ratio of signal to noise in the predicted images.

Visual inspection of the results confirms that the model is able to reconstruct fine details and edges, producing sketches that are coherent and visually similar to the true sketches, even for diverse test images. Some minor variations occur in regions with complex textures or light gradients, typical challenges for image-to-image translation tasks.

The combined loss function integrating MAE and SSIM contributed significantly to this performance, balancing pixel accuracy with structural integrity. This demonstrates the effectiveness of the U-Net architecture with skip connections for image-to-sketch conversion.

Conclusion and Future Work

This study presents the design and implementation of a U-Net convolutional neural network for converting RGB photographs into grayscale sketches. The model demonstrated promising performance, achieving a mean absolute error of 0.0662 and a structural similarity index of 0.72 on the test dataset. These results indicate that the model effectively captures and reconstructs essential structural features, producing sketches that closely resemble the ground truth images. The combined use of pixel-wise (MAE) and perceptual (SSIM) loss functions contributed to the balance between detail accuracy and visual quality.

Despite the model's success, certain limitations were observed. The network occasionally struggled with complex textures and varied lighting conditions, which somewhat affected sketch precision in these regions. Additionally, the dataset's limited diversity and uniform input size constrained the model's generalization ability to broader image domains.

Future research could explore augmenting the dataset with more varied image types and incorporating advanced loss functions or adversarial training to further enhance sketch realism. Improvements in model efficiency would facilitate deployment on resource-constrained devices, expanding real-time application possibilities. Moreover, integrating multi-scale feature extraction or attention mechanisms may better capture intricate details, improving output fidelity.

In conclusion, this work validates the efficacy of U-Net and combined loss functions in artistic-style image translation and provides a foundation for future advancements in automated sketch generation.

References

1. Deep Learning and CNN Fundamentals

Book: Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Link: <https://www.deeplearningbook.org/>

2. U-Net Architecture and Applications

Research Paper: Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation.

PDF: <https://arxiv.org/pdf/1505.04597.pdf>

3. Python Libraries for Deep Learning and Image Processing

Documentation: TensorFlow and Keras provide extensive APIs for building and training CNNs like U-Net. OpenCV and scikit-image offer tools for image preprocessing and evaluation metrics like SSIM and PSNR.

- TensorFlow/Keras: https://www.tensorflow.org/api_docs

- OpenCV: <https://docs.opencv.org/4.x/>

- scikit-image: <https://scikit-image.org/docs/stable/>

4. Image-to-Sketch and Image Translation Research

Paper: Shayak, A., Karki, B., & Shakya, S. (2024). Image Synthesis using U-net: Sketch 2 Image. Nepal Journal of Science and Technology.

PDF:

<https://www.nepjol.info/index.php/NJST/article/download/48266/36319>